

საქართველოს აგრარული უნივერსიტეტი



ს ა კ უ რ ს ო პ რ ო ე ქ ტ ი

4-1

მედიკამენტების შეხსენების სისტემა

შეასრულა:

საინჟინრო-ტექნოლოგიური ფაკულტეტის

მეოთხე კურსის სტუდენტმა ნათია კანდელაკმა

2025 წ.

სარჩევი

1. ანოტაცია	3
2. შესავალი	4
3. ელექტრული სქემა, მისი მუშაობის აღწერა და ანგარიში.....	5
4. კოდის აღწერა.....	8
5. გამოყენებული დეტალების ჩამონათვალი	29
6. გამოყენებული ლიტერატურა/ზმულები ინტერნეტიდან	30

ანოტაცია

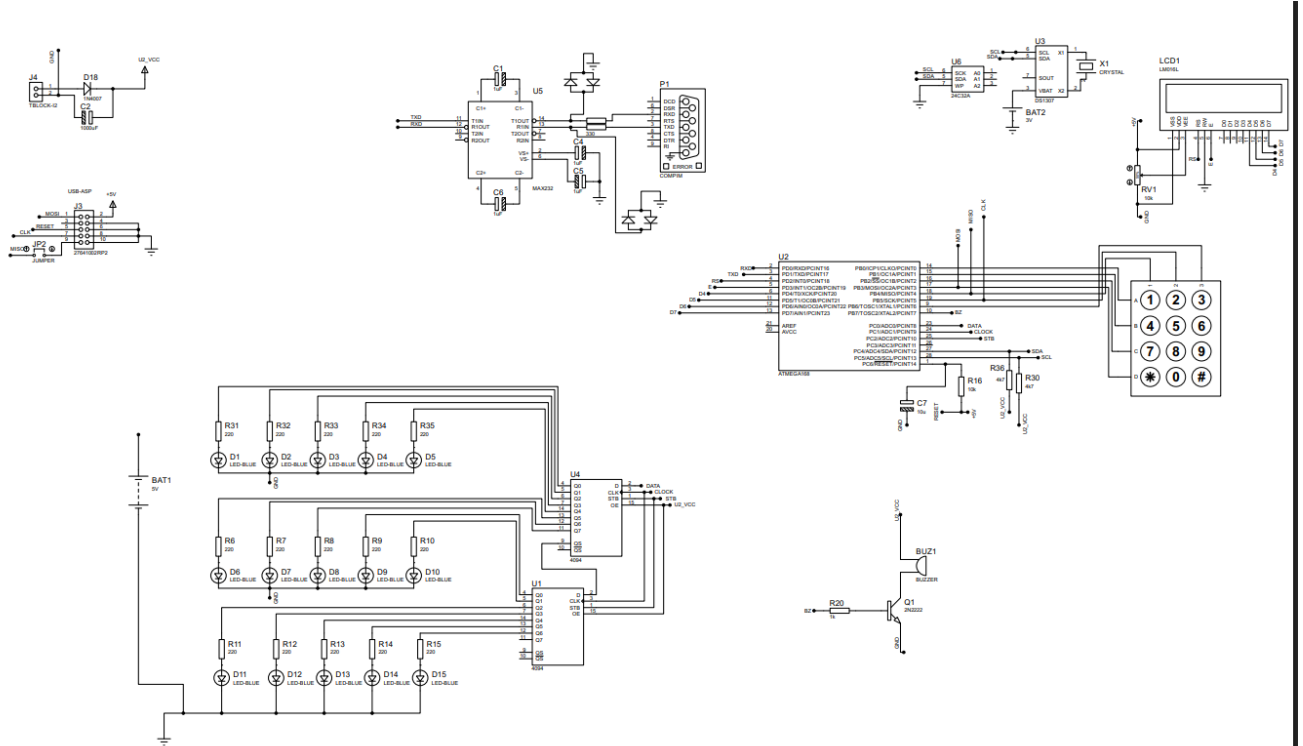
ქრონიკული დაავადებები გლობალურად იზრდება და ქვეყნის ეკონომიკურ და სოციალურ განვითარებაზე ჯანმრთელობის მნიშვნელოვან ტვირთად იქცევა. ბევრ პაციენტს, შეიძლება ყოველდღიურად რამდენიმე მედიკამენტის მიღება სჭირდება, მათი მდგომარეობის მართვა და მედიკამენტების მიღების რეჟიმის მკაცრი დაცვა კი გადაწყვეტი მნიშვნელობისაა ოპტიმალური შედეგების მისაღწევად. ამ პრობლემების გადაჭრა შესაძლებელია მედიკამენტების ავტომატური შეხსენების სისტემით. ავტომატიზირებულ სისტემას შეუძლია უზრუნველყოს თანმიმდევრული შეტყობინებები.

მოწყობილობა ადამიანს შეახსენებს როდის უნდა მიიღოს მედიკამენტი, წინასწარ განსაზღვრულ დროს აუდიო გაფრთხილებების გაგზავნით, თხევად კრისტალურ ეკრანზე გამოსახავს მედიკამენტის კონტეინერის ნომერს და შუქდიოდით მიაწვდის შესაბამის კონტეინერს. იმისათვის რომ მომხმარებელმა თავდაპირველად მიუთითოს მედიკამენტის დრო, გამოყენებულია მატრიცული კლავიატურა, სისტემა იყენებს რეალური დროის საათის (RTC) მოდულს მიმდინარე დროის უწყვეტი კონტროლისთვის და ადარებს მიმდინარე დროს შენახული მედიკამენტების მიღების გრაფიკს. როდესაც მედიკამენტის მიღების დაგეგმილი დრო დადგება, სისტემა ააქტიურებს ხმოვან სიგნალს მომხმარებლისთვის შეტყობინების გასაგზავნად. ამავდროულად, ეკრანი აჩვენებს მედიკამენტის კონტეინერის ნომერს. შუქდიოდები გამოყენებულია კონტეინერების ინდიკატორებად, იმის საჩვენებლად, თუ რომელი კონტეინერი უნდა გახსნას მომხმარებელმა. მედიკამენტის მიღების შემდეგ, მომხმარებელი აჭერს დადასტურების ღილაკს ხმოვანი სიგნალისა და შუქდიოდის გამოსართავად. კონტეინერში მოთავსებულია მედიკამენტები შეფუთვით, რათა თავიდან ავიცილოთ არასწორი მედიკამენტის მიღება და გამოვრიცხოთ ყველა შესაძლო შეცდომა.

შესავალი

მედიკამენტების რეგულარული გამოყენება აუცილებელია მრავალი ადამიანისთვის, განსაკუთრებით ხანდაზმულებისა და ქრონიკული დაავადებების მქონე ადამიანებისთვის. თუმცა, მედიკამენტების მიღების დროის დავიწყებამ შესაძლოა სერიოზული ზიანი გამოიწვიოს. ტრადიციული მკურნალობის მიდგომებში, მედიკამენტების დროული გამოყენების უზრუნველსაყოფად ადამიანის მეხსიერებაზე დაყრდნობა ყოველთვის დაკავშირებული იყო გამოწვევებთან. ციფრული ტექნოლოგიების განვითარებასთან ერთად, ჯანდაცვის სფეროში მრავალი პროცესის ავტომატიზაცია ჩვეულებრივ მოვლენად იქცა. მედიკამენტების მიღების შემხსენებელ მოწყობილობებს, როგორც დამხმარე საშუალებებს, მნიშვნელოვანი პოტენციალი აქვთ პაციენტების მიერ მედიკამენტების მიღების რეჟიმის დაცვის გაუმჯობესებისა და ამით მკურნალობის ეფექტურობის გაზრდის თვალსაზრისით. ეს მოწყობილობები ავალდებულებს პაციენტებს, მიიღონ მედიკამენტები წინასწარ განსაზღვრულ დროს აუდიო ან ვიზუალური გაფრთხილებების გაგზავნით და ხელს უშლის მათ მედიკამენტების დავიწყების ან შეცდომების დაშვებისგან. ასევე მედიკამენტების შემხსენებელი ადამიანებს დაეხმარება თავიდან აიცილონ დოზის გადაჭარბება. ძალიან ბევრი მედიკამენტის მიღება შეიძლება საშიში იყოს და სიკვდილიც კი გამოიწვიოს.

ელექტრული სქემა და მისი მუშაობის აღწერა



ელექტრული სქემის აღწერა:

წრედი მუშაობს 5 ვოლტზე, ეს არის Atmega168-სთვის უსაფრთხო დასაშვები ძაბვა. წრედში გამოყენებულია RTC-ს (რეალური დროის საათი), თხევადკრისტალური ეკრანი, შუქდიოდები, ხმამაღლამოლაპარაკე, ძვრის რეგისტრები შუქდიოდების სამართავად. მომხმარებელს გარედან შეჰყავს სასურველი დრო, კონტინერის ნომერი, მისაღები მედიკამენტის რაოდენობა კლავიატურის მეშვეობით. ეს ინფორმაცია ინახება EEPROM მეხსიერებაში, რომელიც მოთავსებულია რეალური დროის მოდულზე. რეალური დროის საათი ძირითადი კომპონენტის როლს ასრულებს და ზუსტი დროის მითითების უზრუნველყოფით, ის დროზე დაფუძნებული აპლიკაციების განხორციელების საშუალებას იძლევა. მას შეუძლია დროის შენარჩუნება მაშინაც კი, თუ ელექტროენერგია გაითიშება ან მიკროკონტროლერი გადატვირთულია. რეალური დროის საათი იყენებს I2C (ინტეგრირებული წრედის) სერიულ საკომუნიკაციო პროტოკოლს, რომელიც მიკროკონტროლერებს საშუალებას აძლევს დაუკავშირდნენ სხვა მოწყობილობებს.

მას შემდეგ რაც მომხმარებლის მიერ შენახული დრო დაემთხვევა რეალურ დროს პაციენტს გადაეცემა შეტყობინება თხევადკრისტალური ეკრანის, შუქდიოდებისა და ხმამაღლამოლაპარაკის მეშვეობით. ეს შეტყობინება ჩართული იქნება მანამ, სანამ მომხმარებელი არ გათიშავს მას, ანუ მანამ სანამ იგი არ მიიღებს განკუთვნილ მედიკამენტს.

მუშაობის აღწერა:

Atmega168 მიკროკონტროლერი წრედის ცენტრალური მართვის ელემენტის ფუნქციას ასრულებს. ის იღებს რეალური დროის საათის მონაცემებს, ამუშავებს მომხმარებლის მიერ შეყვანილ მონაცემებს, აახლებს ეკრანს და აკონტროლებს შუქდიოდებსა და ხმამაღლამოლაპარაკეს. მიკროკონტროლერი გარე კომპონენტებთან კომუნიკაციას ახდენს შეტანა/გატანის პორტების, მიმდევრული კომუნიკაციის საშუალებით.

რეალური დროის საათის ინტეგრირებული სქემა გამოიყენება დროის ზუსტი აღრიცხვისთვის, მაშინაც კი, როდესაც სისტემის კვება გათიშულია. რეალური დროის საათი განუწყვეტლივ აკონტროლებს წამებს, წუთებს, საათებს, დღეს და თარიღს. მიკროკონტროლერი პერიოდულად კითხულობს დროს და ადარებს მას შეყვანილი მედიკამენტების მიღების გრაფიკს. როდესაც დაპროგრამებული დრო ემთხვევა რეალურ დროს, სისტემა ააქტიურებს შესაბამის შუქდიოდებს და ააქტიურებს ხმამაღლამოლაპარაკეს. თხევადკრისტალური ეკრანი მომხმარებლებს აძლევს მკაფიო ვიზუალურ უკუკავშირს შემდეგ საკითხებზე: მიმდინარე დრო, მედიკამენტების დროის სიგნალიზაცია, მენიუს ნავიგაცია და დაყენება, მომხმარებლის მიერ შერჩეული პარამეტრები. თხევადკრისტალური ეკრანი დაკავშირებულია მიკროკონტროლერთან სტანდარტული პარალელური ინტერფეისის გამოყენებით. მონაცემთა ხაზების და მართვის სიგნალების სერია (როგორცაა RS, E) საშუალებას კლავიატურა გამოიყენება დროის დასაყენებლად, მედიკამენტების მიღების ინტერვალების კონფიგურაციისთვის, მენიუში ნავიგაციისთვის და სიგნალიზაციის დასადასტურებლად.

შუქდიოდების სამართავად გამოყენებულია ძვრის რეგისტრი (CD4094), მიკროკონტროლერის სამი ფეხი PC0, PC1 და PC2 დაკავშირებულია ძვრის რეგისტრთან. მიკროკონტროლერიდან მიმდევრობით 8 ბიტთან ინფორმაციას გადავცემთ ძვრის რეგისტრის data ფეხს, strobe-ის საშუალებით ხდება ინფორმაციის ცვლილების კონტროლი. იმისათვის, რომ მოხდეს 10 შუქდიოდის მართვა, საჭიროა ორი ძვრის რეგისტრი. პირველი ძვრის რეგისტრის ინვერსიული გამოსასვლელი დაკავშირებულია მეორე ძვრის რეგისტრის data შესასვლელზე. U4-ის (CD4094)

პარალელური გამოსასვლელები (Q0-Q7) მიერთებულია შუქდიოდებზე წინაღობებით რათა არ მოხდეს შუქ დიოდების გადაწვა.

$$R = \frac{V - V_{LED}}{I_{LED}}$$

ყველა შუქდიოდს ფერის მიხედვით აქვს სხვადასხვა ძაბვა. ლურჯი 3.1 ვ.

$$R_{\text{ლურჯი}} = \frac{5-3.1}{0.02} = 100 \text{ ომი}$$

სქემაში გამოყენებულია 220 ომი წინააღობები, რადგან თავიდან იქნეს აცილებული ყველანაირი საფრთხე.

ტრანზისტორი Q1 (2N2222) ხმამაღლამოლაპარაკესთან გამოიყენება, რადგან მიკროკონტროლერის გამოსასვლელებს დაახლოებით შეუძლიათ მაქსიმალური დენის მიწოდება 20-40mA. ხმამაღლამოლაპარაკეთა უმეტესობას კი სრული სიმძლავრით მუშაობისთვის 30-80mA სჭირდება, რაც: ძალიან ბევრია მიკროკონტროლერის ფეხისთვის, რათა თავიდან იქნეს აცილებული მიკროკონტროლერის დაზიანების რისკი.

კოდის აღწერა:

კოდი Atmega168 მიკროკონტროლერის გამოყენებით ახდენს რეალური დროის წაკითხვას, ადარებს მას მედიკამენტის მიღების გრაფიკს და საჭირო დროს რთავს შუქდიოდებს და ხმოვან სიგნალს.

ფაილი: shiftreg.c

```
#include "shiftreg.h" //შეიცავს შესაბამის სათაურის ფაილს
static uint8_t led_state[2] = {0, 0}; //შუქდიოდების მდგომარეობის შენახვა
void shiftreg_init(void) {
    DDRC |= (1 << SR_DATA) | (1 << SR_CLOCK) | (1 << SR_STROBE); //DDRC - მონაცემთა
    მიმართულების რეესტრი C პორტისთვის. PC0, PC1, PC2-ის OUTPUT პინებად კონფიგურაცია

    PORTC &= ~(1 << SR_DATA) | (1 << SR_CLOCK) | (1 << SR_STROBE)); //PC0, PC1, PC2-ს დაბალ
    (0V) ნიშნულზე აყენებს, PORTC რეესტრი: აკონტროლებს პორტ C პინების გამომავალ მდგომარეობას

    shiftreg_all_off(); //ყველა ინდიკატორის გასასუფთავებლად გამოძახების ფუნქცია
}
static void shiftreg_update(void) { //გადასცემს led_state[] მასივის შიგთავსს ფიზიკურ ცვლის
    რეესტრებში, ასრულებს მიმდევრულ მონაცემთა გადაცემას, აახლებს ყველა შუქდიოდის გამოსავალს
    ერთდროულად
    PORTC &= ~(1 << SR_STROBE); //SR_STROBE LOW, ამზადებს ცვლის რეესტრებს ახალი
    მონაცემებისთვის
    for (int8_t i = 10; i >= 0; i--) {
        uint8_t byte_index = i / 8; // განსაზღვრავს led_state[]-ის რომელი ბაიტი შეიცავს
        მიმდინარე ბიტს.
        uint8_t bit_index = i % 8; // განსაზღვრავს რომელი ბიტია არჩეულ ბაიტში
        // ბიტის შერჩევა
        uint8_t bit_value;
        if (i < 15) {
            bit_value = (led_state[byte_index] >> bit_index) & 0x01;
        } else {
            bit_value = 0;
        }
        //მონაცემთა პინის დაყენება
        if (bit_value) {
            PORTC |= (1 << SR_DATA);
        } else {
            PORTC &= ~(1 << SR_DATA);
        }
        //საათის პულსი
        PORTC |= (1 << SR_CLOCK);
        _delay_us(1);
        PORTC &= ~(1 << SR_CLOCK);
        _delay_us(1);
    }

    PORTC |= (1 << SR_STROBE);
    _delay_us(1);
    PORTC &= ~(1 << SR_STROBE);
}
```

```

void shiftreg_led_on(uint8_t led_num) {
    if (led_num < 1 || led_num > 10) return; //ჩართავს ერთ შუქდიოდს.
    //მხოლოდ 1-10 შუქდიოდებია მოქმედი
    led_num--;
    //გარდაქმნის ადამიანის ნუმერაციას მასივის ინდექსირებად.
    uint8_t byte_index = led_num / 8;
    uint8_t bit_index = led_num % 8;

    led_state[byte_index] |= (1 << bit_index); //განსაზღვრავს, რომელი ბაიტი და ბიტი უნდა
    შეიცვალოს.

    shiftreg_update(); //არჩეულ ბიტს 1-ზე აყენებს (შუქდიოდი ჩართულია).
}
//იგივეა, რაც led_on() ფუნქცია, მაგრამ ბიტს ასუფთავებს
void shiftreg_led_off(uint8_t led_num) {
    if (led_num < 1 || led_num > 10) return;

    led_num--;

    uint8_t byte_index = led_num / 8;
    uint8_t bit_index = led_num % 8;

    led_state[byte_index] &= ~(1 << bit_index);

    shiftreg_update();
}
//ყველა 16 ბიტის წაშლა → ყველა შუქდიოდი გამორთულია.
void shiftreg_all_off(void) {
    led_state[0] = 0;
    led_state[1] = 0;
    shiftreg_update();
}

void shiftreg_all_on(void) {
    led_state[0] = 0xFF;
    led_state[1] = 0x03;
    shiftreg_update();
}

```

ფაილი: keypad.c

```

#include "keypad.h"
//კლავიატურის განლაგების განმარტება
const char keys[4][3] = {
    {'1', '2', '3'},
    {'4', '5', '6'},
    {'7', '8', '9'},
    {'*', '0', '#'}
};

//ახდენს კლავიატურის პინების ინიციალიზაციას, რათა მიკროკონტროლერმა შეძლოს კლავიშების
დაჭერის წაკითხვა.
void keypad_init(void) {
    DDRB |= (1 << ROW0) | (1 << ROW1) | (1 << ROW2) | (1 << ROW3); // ყველა რიგის პინს
    გამოსავალად აყენებს.
    DDRB &= ~(1 << COL0) | (1 << COL1) | (1 << COL2)); //ყველა სვეტის პინს აყენებს
    შემავალ მნიშვნელობებად.
}

```

`PORTB |= (1 << COL0) | (1 << COL1) | (1 << COL2);` //სვეტის შესასვლელზე რთავს pullup რეზისტორებს. სვეტები ჩვეულებრივ წაიკითხავენ მაღალს, ხოლო ღილაკზე დაჭერისას დაბალს.

`PORTB |= (1 << ROW0) | (1 << ROW1) | (1 << ROW2) | (1 << ROW3);` //თავდაპირველად ყველა რიგს HIGH-ზე აყენებს.

}
//მთავარი ფუნქცია, რომელიც სკანირებს კლავიატურას იმის დასადგენად, თუ რომელი ღილაკია დაჭერილი.

```
char keypad_get_key(void) {
    for (uint8_t row = 0; row < 4; row++) {
        PORTB |= (1 << ROW0) | (1 << ROW1) | (1 << ROW2) | (1 << ROW3); //პირველ
რიგში, ყველა რიგს HIGH მნიშვნელობას ანიჭებს.
        PORTB &= ~(1 << (ROW0 + row)); //ერთდროულად ერთი რიგის დაბალ სიჩქარეზე
მართვას ახდენს. ეს საშუალებას იძლევა დადგინდეს, რომელ სვეტზეა დაჭერილი ღილაკი.
        _delay_us(5);

        for (uint8_t col = 0; col < 3; col++) {
            if (!(PINB & (1 << (COL0 + col)))) { //თუ სვეტში მითითებულია LOW
→ დაჭერილია ამ რიგში/სვეტში არსებული ღილაკი.
                _delay_ms(20);
                if (!(PINB & (1 << (COL0 + col)))) {
                    char key = keys[row][col];
                    // გასაღების აშვების ლოდინი, ციკლი გრძელდება მანამ, სანამ
ღილაკი არ გათავისუფლდება. while (!(PINB & (1 << (COL0 + col)))) {
                        _delay_ms(10);
                    }
                    _delay_ms(50);

                    return key;
                }
            }
        }
    }
    return 0; //ღილაკი არ არის დაჭერილი
}
```

ფაილი: lcd.c

```
#include "lcd.h"
//LCD Enable pin-ის მაღალ მაჩვენებელს აყენებს (ეს LCD-ს ეუბნება, რომ მონაცემები მზადაა).
static void lcd_enable_pulse(void) {
    PORTD |= (1 << LCD_E); // E HIGH
    _delay_us(1);
    PORTD &= ~(1 << LCD_E); // Enable LOW → LCD კითხულობს ინსტრუქციას.
    _delay_us(50); //LCD-ს თითოეული იმპულსის შემდეგ დამუშავების დრო სჭირდება.
}
//ასუფთავებს 4 მონაცემთა ხაზს (D4-D7).
static void lcd_send_nibble(uint8_t nibble) {
    PORTD &= ~(1 << LCD_D4) | (1 << LCD_D5) | (1 << LCD_D6) | (1 << LCD_D7));

    PORTD |= (nibble & 0x0F) << 4; //იღებს ქვედა 4 ნაწილს და ათავსებს D4-D7
წერტილებზე.
    lcd_enable_pulse();
}
```

//გზავნის სრულ 8-ბიტს მნიშვნელობას ორი 4-ბიტის გადამცემის გამოყენებით.

```
static void lcd_send_byte(uint8_t rs, uint8_t byte) {  
    if (rs) {  
        PORTD |= (1 << LCD_RS);          // RS = 0 → ინსტრუქცია  
RS = 1 → სიმბოლოების მონაცემები  
    } else {  
        PORTD &= ~(1 << LCD_RS);        // RS = 0  
    }  
  
    lcd_send_nibble(byte >> 4);  
    lcd_send_nibble(byte);  
    _delay_us(50);  
}
```

```
void lcd_command(uint8_t cmd) {  
    lcd_send_byte(0, cmd); // RS = 0 for commands  
    _delay_ms(2);  
}
```

//გზავნის ერთ სიმბოლოს LCD-ზე.

```
void lcd_putchar(char c) {  
    lcd_send_byte(1, c); // RS = 1 for data  
    _delay_us(50);  
}
```

```
void lcd_init(void) {  
    //gamosasvlelebad  
    DDRD |= (1 << LCD_RS) | (1 << LCD_E) |  
            (1 << LCD_D4) | (1 << LCD_D5) |  
            (1 << LCD_D6) | (1 << LCD_D7);  
  
    _delay_ms(50);  
  
    PORTD &= ~(1 << LCD_RS); // RS = 0  
  
    lcd_send_nibble(0x03);  
    _delay_ms(5);  
  
    lcd_send_nibble(0x03);  
    _delay_us(150);  
  
    lcd_send_nibble(0x03);  
    _delay_ms(1);  
  
    //4-bit mode  
    lcd_send_nibble(0x02);  
    _delay_ms(1);  
  
    lcd_command(LCD_FUNCTION_SET_4BIT);  
  
    lcd_command(LCD_DISPLAY_ON);  
  
    lcd_command(LCD_CLEAR_DISPLAY);  
    _delay_ms(2);  
  
    lcd_command(LCD_ENTRY_MODE_SET);  
}
```

```

void lcd_clear(void) {
    lcd_command(LCD_CLEAR_DISPLAY);
    _delay_ms(2);
}

void lcd_set_cursor(uint8_t row, uint8_t colmn) {
    uint8_t address;
    if (row == 0) {
        address = 0x00 + colmn;
    } else {
        address = 0x40 + colmn;
    }

    lcd_command(0x80 | address);
}

//ბექდავს სტრიქონს სიმბოლო-სიმბოლო, სანამ null ტერმინატორს არ მიაღწევს.
void lcd_print(const char *str) {
    while (*str) {
        lcd_putchar(*str);
        str++;
    }
}

```

ფაილი: main.c

```

//სათაურები და განმარტებები #define F_CPU 8000000UL
#include <avr/io.h> #include <util/delay.h> #include <stdio.h> #include <stdlib.h> #include
<string.h> #include <avr/interrupt.h>

#include "lcd.h" #include "keypad.h" #include "shiftreg.h"

#define BUZZER_PIN 7 // PB7, Buzzer-ის კონფიგურაცია
//ბაზერის კონტროლის ფუნქციები
void buzzer_init(void) { //ზმამაღლამოლაპარაკის ინიციალიზაცია
    DDRB |= (1 << BUZZER_PIN); //მონაცემთა მიმართულების რეგისტრ B-ში მე-7 ბიტის დაყენებით,
PB7-ს OUTPUT-ად აყენებს.
    PORTB &= ~(1 << BUZZER_PIN);} //PORTB-ში მე-7 ბიტის გასუფთავება, საწყისი მდგომარეობის
დაბალზე დაყენება (buzzer გამორთულია)
void buzzer_on(void) {PORTB |= (1 << BUZZER_PIN);} //ბაზერის ჩართვა ოპერაცია: აყენებს PB7 =
HIGH (5V) void buzzer_off(void) {PORTB &= ~(1 << BUZZER_PIN);}
void buzzer_beep(uint16_t duration_ms) {//მოკლე ხმოვანი სიგნალი
    buzzer_on();
    for (uint16_t i = 0; i < duration_ms; i++) {
        _delay_ms(1);
    }buzzer_off();}
//შეტყობინების ფუნქცია
void buzzer_alert(void) {
    for (uint8_t i = 0; i < 3; i++) { //ბაზერის ჩართვა
        buzzer_beep(200);
        _delay_ms(200);
    }
}
//es aris RTC

```

```

//SD serial data (sends and receives), SC serial clock (masteria read da write)
//0x68 0x68 << 1 | 0

#define F_SCL 100000UL //100000 Hz = 100 kHz (სტანდარტული I2C სიჩქარე)
#define Prescaler 1 //TWI prescaler მნიშვნელობა ,გამოიყენება TWBR გამოსათვლელად
#define TWBR_VALUE (((F_CPU / F_SCL) / Prescaler) - 16 ) / 2 // გამოთვლის TWBR (TWI Bit Rate Register) მნიშვნელობას TWBR = 32

//რეალური დროის წაკითხვა/დაყენება
//I2C ინტერფეისის ინიციალიზაცია
void TWI_init(void) {
    TWSR = 0x00; //ასუფთავებს TWI სტატუსის რეგისტრს, წინასწარი სკალერის 1-ზე დაყენებით.
    TWBR = (uint8_t)TWBR_VALUE;
}

uint8_t TWI_get_status(void) {
    return TWSR & 0xF8;
} //TWSR-დან სტატუსის კოდის ამოღება წინასწარი მასშტაბირების ბიტების შენიღბვით (ინარჩუნებს 7-3 ბიტებს, ასუფთავებს 2-0 ბიტებს).

uint8_t TWI_start(void) {
    • TWCR = (1<<TWSTA)|(1<<TWEN)|(1<<TWINT); //SDA ეცემა LOW-ზე მაშინ, როცა SCL არის
    HIGH, - TWSTA = 1 → START
    - TWEN = 1 → ჩართე I2C მოდული
    - TWINT = 1 → გაასუფთავე ინტერფეისის ფლაგი, რომ დაიწყოს ოპერაცია.
    uint16_t timeout = 10000; //იცავს გაჭედვისგან — თუ TWINT არასდროს გახდება 1, არ უნდა
    გაიჭედოს უსასრულოდ.
    while (!(TWCR & (1<<TWINT)) && timeout--);
    if (timeout == 0) return 0;
    if (TWI_get_status() != 0x08) return 0;
    return 1;
}

//I2C STOP condition-ის გენერაცია
void TWI_stop(void) {
    TWCR = (1<<TWSTO)|(1<<TWINT)|(1<<TWEN); //I2C bus ათავისუფლებს ,SDA და SCL ბრუნდება
    HIGH-ზე,I2C ტრანზაქცია დასრულებულია
}

//მონაცემის გაგზავნა
void TWI_write(uint8_t data) {
    TWDR = data; //გზავნის ბაიტს TWI Data Register-ში.
    TWCR = (1<<TWEN)|(1<<TWINT); //იწყებს ბაიტის გაგზავნას
    while (!(TWCR & (1<<TWINT))); //ელოდება გაგზავნის დასრულებას
}

//კითხულობს ბაიტს და აგზავნის ACK-ს → ამით ეუბნება slave-ს „კიდე გამოგზავნე“.
uint8_t TWI_read_ack(void) {
    TWCR = (1<<TWEN)|(1<<TWINT)|(1<<TWEA);
    while (!(TWCR & (1<<TWINT)));
    return TWDR;
}

//მონაცემის წაკითხვა NACK-ით, NACK (Not Acknowledge) გაგზავნა → ბოლო ბაიტი
uint8_t TWI_read_nack(void) {
    TWCR = (1<<TWEN)|(1<<TWINT);
    while (!(TWCR & (1<<TWINT)));
    return TWDR;
}

```

```

#define DS1307_ADDR 0x68
//BCD (Binary-Coded Decimal)-დან ათობითში გადაყვანა მაგ: 0x25 = 25 decimal.
uint8_t bcd_to_decimal(uint8_t val) { return (val >> 4) * 10 + (val & 0x0F); }
uint8_t decimal_to_bcd(uint8_t val) { return ((val / 10) << 4) | (val % 10); }

void DS1307_init(void) {
    TWI_init();
    TWI_start();
    TWI_write((DS1307_ADDR << 1) | 0);
    TWI_write(0x00);
    TWI_start();
    TWI_write((DS1307_ADDR << 1) | 1);
    uint8_t seconds = TWI_read_nack();
    TWI_stop();
    if (seconds & 0x80) {
        TWI_start();
        TWI_write((DS1307_ADDR << 1) | 0);
        TWI_write(0x00);
        TWI_write(seconds & 0x7F); // Clear ClockHalt
        TWI_stop();
        _delay_ms(10);}}

//დროის დაყენება
void DS1307_set_time(uint8_t sec, uint8_t min, uint8_t hour,
uint8_t day, uint8_t date, uint8_t month, uint8_t year)
{
    TWI_start();
    TWI_write((DS1307_ADDR << 1) | 0);
    TWI_write(0x00);
    TWI_write(decimal_to_bcd(sec) & 0x7F); // მაგალითი: sec = 30 decimal_to_bcd(30) =
0x30 = 00110000 (BCD)
    TWI_write(decimal_to_bcd(min));
    TWI_write(decimal_to_bcd(hour) & 0x3F);
    TWI_write(decimal_to_bcd(day));
    TWI_write(decimal_to_bcd(date));
    TWI_write(decimal_to_bcd(month));
    TWI_write(decimal_to_bcd(year));
    TWI_stop();
}

//ეს ფუნქცია RTC-დან კითხულობს HH:MM:SS-ს.
void DS1307_get_time(uint8_t *sec, uint8_t *min, uint8_t *hour,
uint8_t *day, uint8_t *date, uint8_t *month, uint8_t *year) { //იღებს მაჩვენებლებს, რათა
მნიშვნელობები პირდაპირ ჩაიწეროს ცვლადებში მთავარ პროგრამაში.
    TWI_start();
    TWI_write((DS1307_ADDR << 1) | 0);
    TWI_write(0x00);
    TWI_start();
    TWI_write((DS1307_ADDR << 1) | 1);

    *sec = bcd_to_decimal(TWI_read_ack() & 0x7F);
    *min = bcd_to_decimal(TWI_read_ack());
    *hour = bcd_to_decimal(TWI_read_ack());
    *day = bcd_to_decimal(TWI_read_ack());
    *date = bcd_to_decimal(TWI_read_ack());
    *month = bcd_to_decimal(TWI_read_ack());
    *year = bcd_to_decimal(TWI_read_nack());
    TWI_stop();
}

// 24C32N 0x50

```

```

//მიკროკონტროლერი მუდმივად ინახავს შეხსენებებს გარე EEPROM 24C32-ში.
#define EEPROM_24C32_ADDR 0x50
#define MAX_REMINDERS 10
typedef struct {
    uint8_t active;
    uint8_t month;
    uint8_t day;
    uint16_t year;
    uint8_t hour;
    uint8_t minute;
    uint8_t container;
    uint8_t pill_count;
    uint8_t triggered;
} Reminder;
// address არის 16-ბიტის მისამართი (0-4095), data არის ჩასაწერი ბაიტი
void EEPROM_24C32_write_byte(uint16_t address, uint8_t data) {
    TWI_start();
    TWI_write((EEPROM_24C32_ADDR << 1) | 0); // EEPROM-მა ჯერ მისამართის მაჩვენებელი
უნდა მიიღოს ჩაწერის რეჟიმში.
    TWI_write((address >> 8) & 0xFF);
    TWI_write(address & 0xFF);
    TWI_write(data);
    TWI_stop();
    _delay_ms(5);
}
uint8_t EEPROM_24C32_read_byte(uint16_t address) {
    uint8_t data;
    TWI_start();
    TWI_write((EEPROM_24C32_ADDR << 1) | 0);
    TWI_write((address >> 8) & 0xFF);
    TWI_write(address & 0xFF);
    TWI_start();
    TWI_write((EEPROM_24C32_ADDR << 1) | 1);
    data = TWI_read_nack();
    TWI_stop();
    return data;
}
void save_reminder(uint8_t slot, Reminder *rem) { // ფუნქცია იღებს ოპერაციის ნომერს (0-ზე
დაფუძნებული) და შეხსენების სტრუქტურის მიმითითებულს.
    if (slot >= MAX_REMINDERS) return; // არასწორი ოპერაციების ნომრების უარყოფა.
    uint16_t base_addr = slot * 11; // თითოეული შეხსენება EEPROM-ში 11 ბაიტს იკავებს.
    EEPROM_24C32_write_byte(base_addr + 0, rem->active); // ბაიტი 0: აქტიურია თუ არა
შეხსენება.
    EEPROM_24C32_write_byte(base_addr + 1, rem->month);
    EEPROM_24C32_write_byte(base_addr + 2, rem->day);
    EEPROM_24C32_write_byte(base_addr + 3, (rem->year >> 8) & 0xFF);
    EEPROM_24C32_write_byte(base_addr + 4, rem->year & 0xFF); //წელი 16-ბიტისაა →
დაყოფილია ორ ბაიტად.
    EEPROM_24C32_write_byte(base_addr + 5, rem->hour);
    EEPROM_24C32_write_byte(base_addr + 6, rem->minute);
    EEPROM_24C32_write_byte(base_addr + 7, rem->container);
    EEPROM_24C32_write_byte(base_addr + 8, rem->pill_count);
    EEPROM_24C32_write_byte(base_addr + 9, rem->triggered);
}
void load_reminder(uint8_t slot, Reminder *rem) {
    if (slot >= MAX_REMINDERS) return; // არასწორი ოპერაციის უარყოფა.

```

```

uint16_t base_addr = slot * 11; // EEPROM ბაზის მისამართის გამოთვლა.
rem->active = EEPROM_24C32_read_byte(base_addr + 0);
rem->month = EEPROM_24C32_read_byte(base_addr + 1);
rem->day = EEPROM_24C32_read_byte(base_addr + 2);
rem->year = (EEPROM_24C32_read_byte(base_addr + 3) << 8) |
EEPROM_24C32_read_byte(base_addr + 4);
rem->hour = EEPROM_24C32_read_byte(base_addr + 5);
rem->minute = EEPROM_24C32_read_byte(base_addr + 6);
rem->container = EEPROM_24C32_read_byte(base_addr + 7);
rem->pill_count = EEPROM_24C32_read_byte(base_addr + 8);
rem->triggered = EEPROM_24C32_read_byte(base_addr + 9);
}
//იპოვის პირველ თავისუფალ ოპერაციას EEPROM-ში
uint8_t find_empty_slot(void) {
    Reminder temp;
// დროებითი ცვლადი, რომელიც გამოიყენება შეხსენებების წასაკითხად.
    for (uint8_t i = 0; i < MAX_REMINDERS; i++) {
        load_reminder(i, &temp);
        if (temp.active == 0 || temp.active == 0xFF){ //slot ითვლება ცარილად
            return i;
        }
    }
    return 0xFF;
}
//შეხსენების წაშლა
void delete_reminder(uint8_t slot) {
    if (slot >= MAX_REMINDERS) return;
    uint16_t base_addr = slot * 11;
    for(uint8_t i=0;i<11;i++){
        EEPROM_24C32_write_byte(base_addr + i, 0);
    }
}
uint8_t get_datetime(char *buffer) // კითხულობს სრულ დროს DS1307-დან
{
    char disp[17] = "MM/DD/YYYY HH:MM";
    uint8_t pos_map[12] = {0,1,3,4,6,7,8,9,11,12,14,15};
// LCD მბლონი მომხმარებლის ჩანაწერების საჩვენებლად.
    uint8_t index = 0;
    char key;

    lcd_set_cursor(1,0);
    lcd_print(disp);

    while(1)
    {
        key = keypad_get_key();
        if(key >= '0' && key <= '9')
        {
            if(index < 12) // მიიღება მხოლოდ პირველი 12 ციფრი.
            {
                uint8_t digit = key - '0'; // ASCII → რიცხვის კონვერტაცია.
                uint8_t allow = 1; // კონტროლის დაშვება, თუ ციფრი სწორია.
                switch(index) {
                    case 0:
                        if(digit > 1) allow = 0; // თვის ათეულები (ინდექსი 0)
                        break;
                    case 1: ...

```

```

        if(allow) { // თუ ციფრი დაშვებულია: მოათავსეთ ის LCD-ზე
            uint8_t lcdpos = pos_map[index];
            disp[lcdpos] = key;
            buffer[index] = key;
            index++;
            lcd_set_cursor(1, lcdpos);
            lcd_putchar(key);
        } else {
            buzzer_beep(50);
        }
    }
    _delay_ms(180);
}
else if(key == '*') // მომხმარებელი აჭერს '*' -ს (წაშლა)
{
    if(index > 0)
    {
        index--;
        uint8_t lcdpos = pos_map[index];
        disp[lcdpos] = (pos_map[index] == 2 || pos_map[index] == 5 ||
            pos_map[index] == 10 || pos_map[index] == 13)
            ? '/' : ' ';

        // ერთი ციფრით უკან დაბრუნდით
        // ციფრის წაშლა და სლემის ან ინტერვალის აღდგენა.
        if(lcdpos == 0 || lcdpos == 1) disp[lcdpos] = 'M';
        if(lcdpos == 3 || lcdpos == 4) disp[lcdpos] = 'D';
        if(lcdpos >= 6 && lcdpos <= 9) disp[lcdpos] = 'Y';
        if(lcdpos == 11 || lcdpos == 12) disp[lcdpos] = 'H';
        if(lcdpos == 14 || lcdpos == 15) disp[lcdpos] = 'M';

        lcd_set_cursor(1, lcdpos);
        lcd_putchar(disp[lcdpos]);
        buffer[index] = 0;
    }
    else
    {
        return 0;
    }
    _delay_ms(180);
}
else if(key == '#') // მომხმარებელი აჭერს '#' -ს (დადასტურება)
{
    if(index == 12)
    {
        buffer[12] = 0;
        return 1;
    }
    else
    {
        buzzer_beep(150);
    }
    _delay_ms(180);
}
}
}
} // რიცხვითი მნიშვნელობა თარიღის ფორმატირების გარეშე
uint8_t get_digits(char *buffer, uint8_t max_digits) {
    uint8_t count = 0;
    char key;

```

```

while (1) {
    key = keypad_get_key(); //ელოდება ღილაკის დაჭერას.
    if (key >= '0' && key <= '9') {
        if (count < max_digits) {
            buffer[count++] = key;
            lcd_putchar(key);
        }
        _delay_ms(200);
    }
    else if (key == '*') ...
}
uint8_t get_pill_count(void) {
    char buffer[5] = {0}; //4-ნიშნა რიცხვის შესანახი ბუფერი ("0000" max)
    uint8_t count = 0; //მიანიშნებს რამდენი სიმბოლოა შეყვანილი
    uint8_t has_decimal = 0;
    char key;

    while (1) {
        key = keypad_get_key();

        if (key >= '0' && key <= '9') {
            if (count < 4) {
                buffer[count++] = key;
                lcd_putchar(key);
            }
            _delay_ms(200);
        }
        else if (key == '*') {...
    }
    buffer[count] = '\0'; //სტრიქონის ბოლოში ვამატებთ '\0' — ბუფერი ხდება სტრინგი
    uint8_t whole = 0;
    uint8_t decimal = 0;
    if (has_decimal) { //თუ გამოყენებულია წერტილი (ანუ შეყვანილია X.Y ფორმატი)
        for (uint8_t i = 0; i < count; i++) {
            if (buffer[i] == '.') {
                if (i + 1 < count) {
                    decimal = buffer[i + 1] - '0';
                }
                buffer[i] = '\0';
                break;
            }
        }
    }
    whole = atoi(buffer); //მთელი ნაწილის გადაყვანა რიცხვად

    if (decimal >= 5) decimal = 5; //0-4 → 0 | 5-9 → 5 (ანუ მივიღოთ მხოლოდ .0 ან .5)
    else decimal = 0;
    } else {
        whole = atoi(buffer);
    }
}

char get_menu_choice(void) {
    char key;
    while (1) {
        key = keypad_get_key();

        if (key >= '1' && key <= '4') {
            _delay_ms(200);

```

```

        return key;
    }
    _delay_ms(10);
}

void set_rtc_time(void) {
    char input_buffer[15];    //ზუფერი, სადაც შეინახება შეყვანილი თარიღი/დრო

    lcd_clear();
    lcd_print("Set RTC Time");
    _delay_ms(1000);

    lcd_clear();
    lcd_print("MM/DD/YYYY HH:MM");
    lcd_set_cursor(1, 0);

    if (!get_datetime(input_buffer)) {
        buzzer_beep(100);
        return;
    }

    //ზუფერიდან თარიღისა და დროის ციფრების ამორთმევა
    uint8_t month = (input_buffer[0] - '0') * 10 + (input_buffer[1] - '0');
    uint8_t date = (input_buffer[2] - '0') * 10 + (input_buffer[3] - '0');
    uint16_t year = (input_buffer[4] - '0') * 1000 + (input_buffer[5] - '0') * 100 +
        (input_buffer[6] - '0') * 10 + (input_buffer[7] - '0');
    uint8_t hour = (input_buffer[8] - '0') * 10 + (input_buffer[9] - '0');
    uint8_t minute = (input_buffer[10] - '0') * 10 + (input_buffer[11] - '0');

    //მიღებული მონაცემების სისრულის შემოწმება (ნებადართული დიაპაზონები)
    if (month < 1 || month > 12 || date < 1 || date > 31 || hour > 23 || minute > 59)
    {
        lcd_clear();
        lcd_print("Invalid input!");
        buzzer_beep(500);
        _delay_ms(1500);
        return;
    }

    uint8_t year_2digit = year % 100; //DS1307 იყენებს მხოლოდ ორნიშნაან წელს (00-99)
    uint8_t day_of_week = 1;

    DS1307_set_time(0, minute, hour, day_of_week, date, month, year_2digit);
    _delay_ms(100);

    //კონტროლისთვის ვკითხულობთ უკან RTC-დან, სანახავად სწორად ჩაიწერა თუ არა
    uint8_t sec_r, min_r, hour_r, day_r, date_r, month_r, year_r;
    DS1307_get_time(&sec_r, &min_r, &hour_r, &day_r, &date_r, &month_r, &year_r);
    char display[17];
    lcd_clear();
    lcd_print("RTC Set!");
    lcd_set_cursor(1, 0);
    sprintf(display, "%02d/%02d/%02d %02d:%02d", month_r, date_r, year_r, hour_r,
min_r); //დაბეჭდვის ფუნქციები...

    lcd_print(display);

    buzzer_beep(200);
    _delay_ms(2500);
}

void add_reminder(void) {

```

```

char input_buffer[15];

uint16_t year;
uint8_t month, day, hour, minute, container, pill_count;

lcd_clear();
lcd_print("MM/DD/YYYY HH:MM");
lcd_set_cursor(1, 0);
if (!get_datetime(input_buffer)) {
    buzzer_beep(100);
    return;
}
//შეყვანილი თარიღისა და დროის დამუშავება
month = (input_buffer[0] - '0') * 10 + (input_buffer[1] - '0');
day = (input_buffer[2] - '0') * 10 + (input_buffer[3] - '0');
year = (input_buffer[4] - '0') * 1000 + (input_buffer[5] - '0') * 100 +
(input_buffer[6] - '0') * 10 + (input_buffer[7] - '0');
hour = (input_buffer[8] - '0') * 10 + (input_buffer[9] - '0');
minute = (input_buffer[10] - '0') * 10 + (input_buffer[11] - '0');
//თვეების მიხედვით დღეთა მაქსიმუმი
uint8_t max_day = 31;
if (month == 2) max_day = 29;
else if (month == 4 || month == 6 || month == 9 || month == 11) max_day = 30;
//კონტეინერის (უჯრის) არჩევა 1-10.... ჩაწერა
if (day > max_day) {
    lcd_clear();
    lcd_print("Invalid day!");
    buzzer_beep(500);
    _delay_ms(2000);
    return;
}
lcd_clear();
lcd_print("Container (1-10):");
lcd_set_cursor(1, 0);

uint8_t index = 0;
char key;
input_buffer[0] = 0;
input_buffer[1] = 0;

while (1) {
    key = keypad_get_key();

    if (key >= '0' && key <= '9') {
        uint8_t digit = key - '0';
        uint8_t allow = 0;
        ... ზუსტად იგივე ლოგიკა
//კონტეინერის ნომერი რეალურ რიცხვად
        container = atoi(input_buffer);
        lcd_clear();
//აბეზის რაოდენობის შეყვანა
        lcd_print("Pills (*.=.):");
        lcd_set_cursor(1, 0);

        pill_count = get_pill_count();

        if (pill_count == 0xFF) {

```

```

        buzzer_beep(100);
        return;
    }

    if (pill_count == 0 || pill_count > 16) {
        lcd_clear();
        lcd_print("Invalid amount!");
        buzzer_beep(500);
        _delay_ms(1500);
        return;
    }

    uint8_t rem = find_empty_slot();
    if (rem == 0xFF) {
        lcd_clear();
        lcd_print("Memory Full!");
        buzzer_beep(500);
        _delay_ms(2000);
        return;
    }
    //ახალი Reminder სტრუქტურის შევსება
    Reminder new_reminder;
    new_reminder.active = 1;
    new_reminder.month = month;
    new_reminder.day = day;
    new_reminder.year = year;
    new_reminder.hour = hour;
    new_reminder.minute = minute;
    new_reminder.container = container;
    new_reminder.pill_count = pill_count;
    new_reminder.triggered = 0;

    save_reminder(rem, &new_reminder);

    lcd_clear();
    lcd_print("Saved!");
    buzzer_beep(200);
    _delay_ms(1000);
}

```

//ყველა შენახული reminder-ის ნახვა ერთად, ფუნქცია ითვლის რამდენი შეხსენება არის აქტიური, გადის ყველა სლოტს EEPROM-ში, თუ active == 1, ითვლის როგორც არსებულს. თუ არცერთი არ არის — აჩვენებს "No reminders". ტვირთავს EEPROM-დან (load_reminder), თუ reminder აქტიურია, აჩვენებს LCD-ზე

```

void view_reminders(void) {
    Reminder temp;
    char display[17];
    uint8_t total_reminders = 0;

    for (uint8_t i = 0; i < MAX_REMINDERS; i++) {
        uint16_t addr = i * 11;
        uint8_t active = EEPROM_24C32_read_byte(addr);
        if (active == 1) {
            total_reminders++;
        }
    }
}

```

```

    if (total_reminders == 0) {
        lcd_clear();
        lcd_print("No reminders");
        _delay_ms(2000);
        return;
    }

    uint8_t current = 0;
    for (uint8_t i = 0; i < MAX_REMINDERS; i++) {
        load_reminder(i, &temp);

        if (temp.active == 1) {
            current++;

            lcd_clear();
            sprintf(display, "%02d/%02d/%04d", temp.month, temp.day, temp.year);
            lcd_print(display);

            lcd_set_cursor(1, 0);
            sprintf(display, "%02d:%02d C%d:", temp.hour, temp.minute,
temp.container);
            lcd_print(display);

            if (temp.pill_count % 2 == 0) {
                sprintf(display, "%d", temp.pill_count / 2);
            } else {
                sprintf(display, "%d.5", temp.pill_count / 2);
            }
            lcd_print(display);

            lcd_set_cursor(1, 11);
            sprintf(display, "[%d/%d]", current, total_reminders);
            lcd_print(display);

            char key = 0;
            while (1) {
                key = keypad_get_key();

                if (key == '*') {
                    buzzer_beep(100);
                    _delay_ms(200);
                    return;
                }
                else if (key == '#') {
                    buzzer_beep(50);
                    _delay_ms(200);
                    break;
                }

                _delay_ms(50);
            }
        }
    }
    lcd_clear();
    lcd_print("End of list");
    lcd_set_cursor(1, 0);
    lcd_print("Press * to exit");

```

```

while (1) {
    char key = keypad_get_key();
    if (key == '*') {
        buzzer_beep(100);
        _delay_ms(200);
        return;
    }
    _delay_ms(50);
}
}

```

//ფუნქცია მომხმარებელს აძლევს საშუალებას წაშალოს კონკრეტული reminder EEPROM-დან.
ამოწმებს შეყვანილ ნომერს, თუ შეყვანილი ოპერაცია არის 1-MAX_REMINDERS-ის დიაპაზონს გარეთ →
აჩვენებს "Invalid slot!"

```

void delete_reminder_menu(void) {
    char input_buffer[5];

    lcd_clear();
    lcd_print("Delete slot #.");
    lcd_set_cursor(1, 0);

    if (!get_digits(input_buffer, 2)) {
        buzzer_beep(100);
        return;
    }

    uint8_t slot = atoi(input_buffer);

    if (slot < 1 || slot > MAX_REMINDERS) {
        lcd_clear();
        lcd_print("Invalid slot!");
        buzzer_beep(500);
        _delay_ms(1500);
        return;
    }
    slot--;
    Reminder temp;
    load_reminder(slot, &temp);

    if (temp.active != 1) {
        lcd_clear();
        lcd_print("reminder empty");
        buzzer_beep(500);
        _delay_ms(1500);
        return;
    }
    delete_reminder(slot);
    lcd_clear();
    lcd_print("Deleted!");
    buzzer_beep(200);
    _delay_ms(1500);
}

volatile uint8_t rtc_tick_flag = 0;
volatile uint8_t last_min_seen = 0xFF;
//ყველა Compare Match-ზე (რაც ჩვეულებრივ 1 წამში ერთხელ ხდება) იგზავნება სიგნალი →
rtc_tick_flag = 1.
ISR(TIMER1_COMPA_vect) {

```

```

        rtc_tick_flag = 1;
    }
//ეს ფუნქცია პასუხისმგებელია როდესაც დადგება შეხსენების დრო, მომხმარებელს აცნობოს,
აჩვენოს დეტალები და დაელოდოს დადასტურებას.
void trigger_reminder(uint8_t slot, Reminder *r) {
    buzzer_alert();
    shiftreg_led_on(r->container);
    lcd_clear();
    lcd_print("REMINDER!");
    lcd_set_cursor(1, 0);
    char buf[17];
    sprintf(buf, "%02d/%02d/%04d", r->month, r->day, r->year);
    lcd_print(buf);

    _delay_ms(2000);

    lcd_clear();
    lcd_print("Take pills from:");
    lcd_set_cursor(1, 0);
//If pill_count ლუწია → მთელი აბების რაოდენობა, If კენტია → 0.5 აბიც (ანუ 1/2 აბის საჭიროება)
    if (r->pill_count % 2 == 0) {
        sprintf(buf, "C%d: %d pills", r->container, r->pill_count / 2);
    } else {
        sprintf(buf, "C%d: %d.5 pills", r->container, r->pill_count / 2);
    }
    lcd_print(buf);
    _delay_ms(2000);
    lcd_clear();
    lcd_print("Press # when");
    lcd_set_cursor(1, 0);
    lcd_print("pills taken");
    uint16_t beep_counter = 0;
    char key = 0;

    while (key != '#') {... იმავე ლოგიკას ვიყენებ
}

#define MAX_PENDING 4
uint8_t pending_reminders[MAX_PENDING];
uint8_t pending_count = 0;
void add_pending_reminder(uint8_t slot) {
    if (pending_count < MAX_PENDING) {
        pending_reminders[pending_count++] = slot;
    }
}
void process_pending_reminders(void) {
    while (pending_count > 0) {
        uint8_t slot = pending_reminders[0];

        for (uint8_t i = 0; i < pending_count - 1; i++) {
            pending_reminders[i] = pending_reminders[i + 1];
        }
        pending_count--;

        Reminder tmp;
        load_reminder(slot, &tmp);
        trigger_reminder(slot, &tmp);
    }
}

```

```

    }
}
//ეს ბლოკი ქმნის პატარა რიგს ("queue"), სადაც ინახება reminder-ები, რომლებიც უნდა
ამოშავდეს, მაგრამ იმ წამს ვერ ხდება მათი გამოძახება.
void reminders_check_and_trigger(void) {
    uint8_t sec_r, min_r, hour_r, day_r, date_r, month_r, year_r;
    DS1307_get_time(&sec_r, &min_r, &hour_r, &day_r, &date_r, &month_r, &year_r);
    uint16_t full_year = 2000 + year_r;

    if (min_r == last_min_seen) return;
    last_min_seen = min_r;

    Reminder tmp;
    for (uint8_t slot = 0; slot < MAX_REMINDERS; slot++) {
        load_reminder(slot, &tmp);
        if (tmp.active == 1 && tmp.triggered == 0) {
            if (tmp.year == full_year &&
                tmp.month == month_r &&
                tmp.day == date_r &&
                tmp.hour == hour_r &&
                tmp.minute == min_r) {
                add_pending_reminder(slot);
            }
        }
    }
    process_pending_reminders();
}
//ეს ფუნქცია აყენებს Timer1-ს, რომ მოხდეს 1 წამში ერთხელ Compare Match Interrupt
void timer1_init(void) {
    TCCR1A = 0;
    TCCR1B = 0;
    TCNT1 = 0;
    //CTC რეჟიმის ჩართვა (Clear Timer on Compare Match)
    TCCR1B |= (1 << WGM12);
    //Timer1 დააყენე ისე, რომ 7812 ჩათვლამდე ითვალოს.
    OCR1A = 7812;
    //Prescaler-ის დაყენება (1024)
    TCCR1B |= (1 << CS12) | (1 << CS10);
    TIMSK1 |= (1 << OCIE1A);
}
////////////////////////////////////

int main(void) {
    // სისტემის ინიციალიზაცია
    lcd_init();
    keypad_init();
    buzzer_init();
    DS1307_init();
    shiftreg_init();
    timer1_init();
    sei();
    //ეკრანზე გამოდის თავდაპირველი მესიჯი
    lcd_clear();
    lcd_print("Medication");
    lcd_set_cursor(1, 0);
    lcd_print("Reminder System");
    buzzer_beep(150);
}

```

```

    _delay_ms(2000);

    uint8_t sec, min, hour, day, date, month, year;
    uint8_t last_sec = 0xFF;
    lcd_clear();

    while (1) {
//RTC-დან რეალური დროის წაკითხვა
        DS1307_get_time(&sec, &min, &hour, &day, &date, &month, &year);
//თუ წამი შეიცვალა → განახლდეს ეკრანი
        if (sec != last_sec) {
            char date_str[17];
            char time_str[17];
            sprintf(date_str, "%02d/%02d/20%02d", month, date, year);
            sprintf(time_str, "%02d:%02d:%02d", hour, min, sec);
//ბეჭდავს თარიღს და დროს ეკრანზე
            lcd_set_cursor(0, 0);
            lcd_print(date_str);
            lcd_set_cursor(1, 0);
            lcd_print(time_str);

            last_sec = sec;
        }
//ამოწმებს ნებისმიერი ღილაკის დაჭერა ხდება თუ არა
        char key = keypad_get_key();
        if (key != 0) {
            buzzer_beep(30);
            break;
        }
        _delay_ms(30);
    }

    while (1) {
//reminder trigger-ის შემოწმება
        if (rtc_tick_flag) {
            cli();
            rtc_tick_flag = 0;
            sei();
            reminders_check_and_trigger();
        }
//მთავარი მენიუს გამოტანა ეკრანზე
        lcd_clear();
        lcd_print("1.Time 2.Add");
        lcd_set_cursor(1, 0);
        lcd_print("3.View 4.Del *RTC");
//ელოდება სანამ მომხმარებელი რამეს დააჭერს
        char choice = 0;
//მუდმივად ამოწმებს rtc_tick_flag-ს (შეხსენების დრო ხომ არ დადგა)
        while (choice == 0) {
            if (rtc_tick_flag) {
                cli();
                rtc_tick_flag = 0;
                sei();
                reminders_check_and_trigger();
            }
            char key = keypad_get_key();
            if ((key >= '1' && key <= '4') || key == '*') {

```

```

        choice = key;
        buzzer_beep(50);
    }
    _delay_ms(10);
}
//თუ მომხმარებელმა დააჭირა * მენიუდან გადადის ისევ რეალური დროის ჩვენებაზე
if (choice == '*') {
    uint8_t sec, min, hour, day, date, month, year;
    uint8_t last_sec = 0xFF;
    uint8_t last_min = 0xFF;
    uint8_t last_date = 0xFF;

    lcd_clear();

    while (1) {
        if (rtc_tick_flag) {
            cli(); // Interrupt-ების დროებით გათიშვა
            rtc_tick_flag = 0; // flag-ის განულება
            sei(); // Interrupt-ების ჩართვა
            reminders_check_and_trigger();
            lcd_clear();

//LCD refresh reset

            last_sec = 0xFF;
            last_min = 0xFF;
            last_date = 0xFF;
        }

        DS1307_get_time(&sec, &min, &hour, &day, &date, &month,
&year);

        if (date != last_date || min != last_min) {
            if (date != last_date) {
                char date_str[17];
                sprintf(date_str, "%02d/%02d/20%02d", month,
date, year);

                lcd_set_cursor(0, 0);
                lcd_print(date_str);
                last_date = date;
            }

            if (min != last_min) {
                char time_str[9];
                sprintf(time_str, "%02d:%02d:", hour, min);
                lcd_set_cursor(1, 0);
                lcd_print(time_str);
                last_min = min;
            }
        }

        if (sec != last_sec) {
            char sec_str[3];
            sprintf(sec_str, "%02d", sec);
            lcd_set_cursor(1, 6);
            lcd_print(sec_str);
            last_sec = sec;
        }

        char key = keypad_get_key();

```

```

        if (key != 0) {
            buzzer_beep(30);
            break;
        }

        _delay_ms(30);
    }
    continue;
}

switch(choice) {
    case '1': set_rtc_time(); break;
    case '2': add_reminder(); break;
    case '3': view_reminders(); break;
    case '4': delete_reminder_menu(); break;
}
}
}

```

გამოყენებული დეტალების ჩამონათვალი

1. წინაღობა : 1k (1), 4k7(2), 220 (10), 10k(1)
2. ცვლადი რეზისტორი 10K
3. მიკროკონტროლერი Atmega168 (1)
4. ტრანზისტორი 2N2222
5. ძვრის რეგისტრი - CD4094 (2)
6. რეალური დროის მოდული DS1307
7. თხევად კრისტალური ეკრანი
8. 4x3 ალფანუმერული კლავიატურა
9. კონდენსატორი 10uF (1)
10. 5V ხმამაღლამოლაპარაკე
11. კონექტორი 10 ფეხიანი
12. სამონტაჟო დაფა
13. კვების წყარო

გამოყენებული ლიტერატურა/ზმულები ინტერნეტიდან

1. <https://www.ti.com/lit/ds/symlink/cd4049ub.pdf>
2. https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-9365-Automotive-Microcontrollers-ATmega88-ATmega168_Datasheet.pdf
3. <https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf>
4. <https://www.ti.com/lit/ds/symlink/tca8418.pdf?ts=1763898242465>